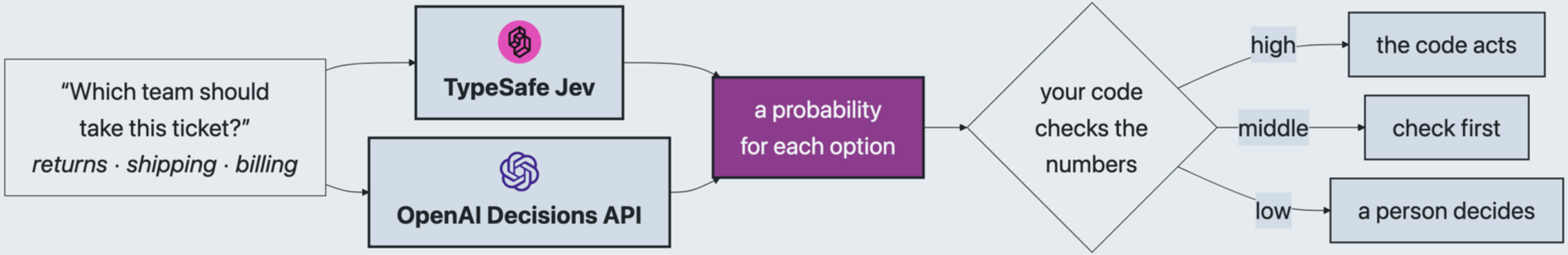
Decision AI Models:
Complete Tutorial on TypeSafe Jev and the OpenAI Decisions API

Learn both from zero, then build real projects with them. For engineers.

Instructor: Roni Das · systemdesign.academy · YouTube: Total Technology Zonne



github.com/ronidas39/jev-system-one-course

Meet Jev and the Decisions API

Both run on the company's computers. Your program calls them over the internet, through an API.

TYPESAFE JEV

READS TEXT

Jev

an AI model from TypeSafe, San Francisco

`released`	15 September 2026
`you send`	text, plus questions with fixed options
`you get`	a probability for each option, never a sentence
`reads`	**text only**
`example`	a support ticket: which team should take it? returns, shipping or billing

OPENAI DECISIONS API

READS TEXT AND PHOTOS

Decisions API

an API from OpenAI, the makers of ChatGPT

`runs on`	OpenAI's model GPT-6 Luna
`released`	6 October 2026, as a public beta
`you send`	text or photos, plus questions with fixed options
`you get`	a probability for each option, never a sentence
`reads`	**text and photos**
`example`	an egg photo: does the shell look cracked?

Who it is for, and what you will learn

Backend
Sort support tickets by team and urgency.

Full-stack, frontend
Check a chat model's reply before your user sees it.

Data engineering
Find duplicate company records. Build a knowledge graph.

You work with photos
Send egg photos to the Decisions API. Find the cracked ones.

ML and AI engineering
Help an AI agent pick a tool. Ask first when money moves.

DevOps, SRE, platform
Pin a model version. Read its errors. See speed and cost.

QA and test
Test a company's claims with small, real scripts.

Architects, tech leads
Pick Jev, the Decisions API or a chat model for a task.

Students, switchers
Basic Python is enough. Five Jev projects, a refund desk, a lane race, a web app.

BY THE END, YOU CAN DO THESE NINE THINGS

1. **You can** explain what an LLM is, and what a System One model does instead. `the idea`
2. **You can** ask Jev its three question types, many in one call. `question types`
3. **You can** turn the numbers into an action: act, check first, or ask a person. `cut-offs`
4. **You can** keep your API keys out of your code, and call Jev and the Decisions API. `setup`
5. **You can** test a company's claims about its model with your own scripts. `fair tests`
6. **You can** build four small Jev projects and a knowledge-graph project. `projects`
7. **You can** use the Decisions API on text, photos and video frames, set cut-offs, send the unsure middle to a person, and build a refund desk and a lane race. `Decisions API`
8. **You can** choose Jev, the Decisions API or a chat model, or use Jev and a chat model together. `side by side`
9. **You can** put all three question types into a small web app that uses both. `web app`

Before you start, and the 14 topics

github.com/ronidas39/jev-system-one-course

`git clone` → `pip install -r requirements.txt` → `.env: TYPESAFE_API_KEY, OPENAI_API_KEY`

Mac and Windows steps: the README

YOU NEED

- Basic Python: run a script, read a function
- A terminal, a code editor, Python 3.12+
- Git, to copy the course code
- A TypeSafe account and API key
- An OpenAI key with a little credit
  One Jev project can use it. Every Decisions API lesson needs it.

YOU DO NOT NEED

- ⊘ Machine-learning maths
- ⊘ A GPU (a graphics card for AI)
- ⊘ Earlier AI experience
- ⊘ To train a model

What it costs: very little. Several scripts print their own cost as they run, so you see it on your screen.

THE 14 TOPICS

1. Welcome, Jev and the Decisions API, and your first call `first code`
2. Why chat models are awkward inside code
3. Three kinds of question, and many at once
4. How sure is it, and what should your code do
5. Weak spots, and where Jev helps
6. Who built Jev, what they claim, and a fair test
7. Three Jev projects, step by step
8. The big project: duplicate companies to a knowledge graph
9. OpenAI's Decisions API and your first call
10. Its three question types, and refusals
11. Photos: grading eggs and a refund desk, step by step
12. The lane race, step by step
13. Side by side on the same 300 tickets
14. The web app, tab by tab

The same call, from Python


```
(.venv) handson $ python 01_first_call.py
model that answered           jev-1.13.0
urgency (noul)                0.98
input tokens                  302
output tokens                 21
cost (USD)                    0.00001268
time on this machine (s)      0.354
```

How a chat model writes: tokens, one at a time

1 The finish-the-sentence game

a friend: "Thank you very..."

you: "much."

you have heard it many times

a large language model (LLM) plays the same game



a huge amount of text

an example: the usual next piece comes first

2 Text is cut into tokens, and you pay per token

input tokens: what you send in

Thank	you	for	the	un	believ	ably	kind	words	!

you pay per 1 million input tokens

→ the model →

output tokens: what it writes back

You	are	welcome	.

per 1 million output tokens: **usually more**

"Unbelievably" is cut into three pieces. An example cut, not from a real tokenizer.

3 A chat model writes one token, then the next



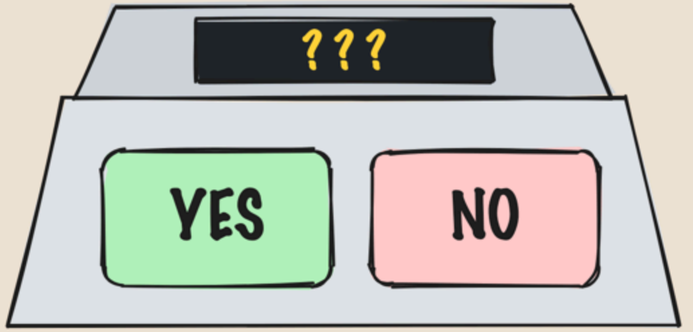
Each token waits for the one before it. This is called autoregressive.

A chatty answer, and four problems for code

You, the owner, ask the shop worker: “Is this customer angry? Yes or no?”

The worker: “Well, it's hard to say. The customer seems a bit upset, but they were polite. I would say mostly yes, maybe.”

the billing machine cannot read a sentence



Four problems when software asks a chat model small questions

My summary of a table in TypeSafe's launch post

1 · Wrong shape

Code needs true or false, a number, or one name from a list. It gets text.

2 · Slow for code

Seconds per answer add up when code asks thousands of questions.

3 · Costs too much

It can cost more than the small decision is worth.

4 · Which to doubt?

It is hard to know which answers to doubt.

Parsing: reading the text and turning it into values your code can use

Just ask for JSON, and which answers to doubt

1 "Just ask the model to answer in JSON."

with structured output, this works well

strict structured output: one of your three

```
{"team": "billing"}
```

✓ a path exists

without it: a fourth name that sounds reasonable

```
{"team": "partnerships"}
```

✗ no path for it

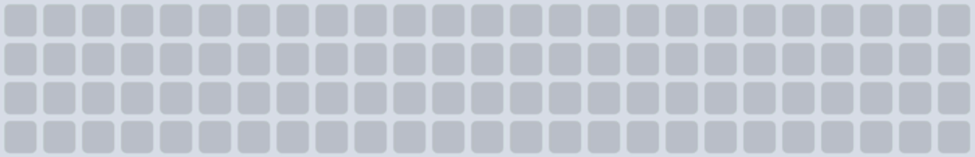
no matching if

your code has a path for
if team == "billing"
if team == "technical"
if team == "sales"

2 Right 95 times in 100, but which 5?

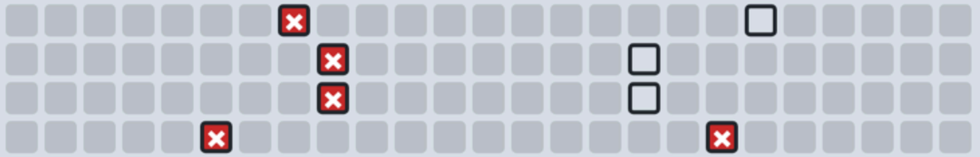
`TypeSafe says` "If a model can do a task 95% of the time but doesn't say when it's in the 5%, it can't automate that task."

WITHOUT A DOUBT SIGNAL — **a person checks all 100**



95 right and 5 wrong, but they all look the same.

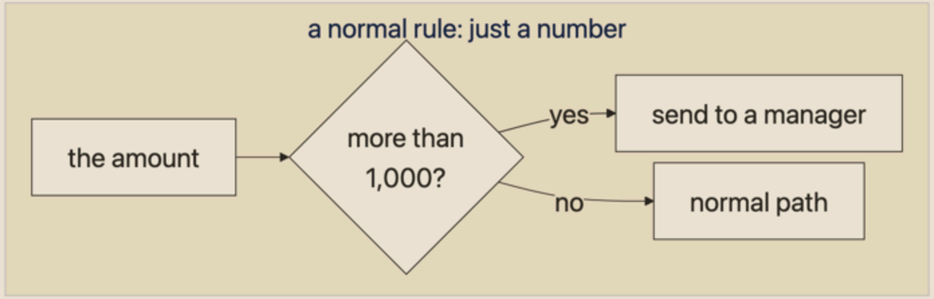
WITH A DOUBT SIGNAL — **a person checks only 8**

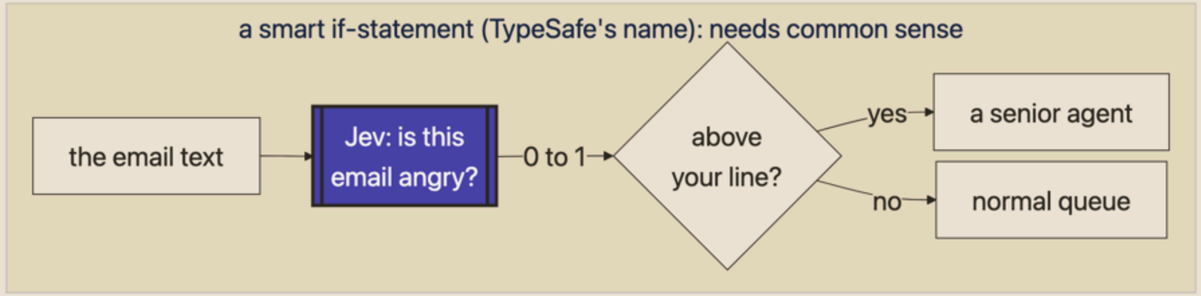


8 flagged as unsure: red are wrong, white are right. Ideally all 5 wrong ones are flagged.

With Jev, you write the options. Each option comes back with a probability.

Fixed options, and the smart if-statement





a chatbot: a sentence to parse (an example)
"This sounds like a returns issue, but the double charge may be billing..."

fixed options: you write them, the AI gives a probability for each

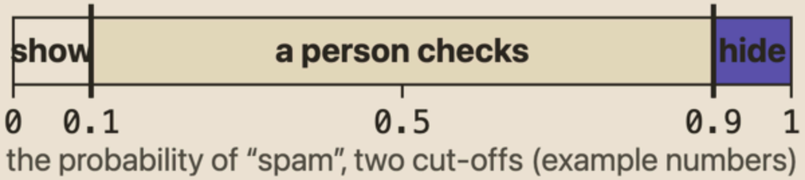
`returns` `shipping` `billing`

FOUR GAINS FOR CODE

1. No text to parse: your code gets values
2. Always one of your options
3. A number, so you can set a cut-off
4. It writes no text: you pay for what it reads

```
# a plain number
if amount > 1000:
# common sense, as a number
if p_angry > ANGRY_LINE:
```

"Is this review spam?"



the probability of "spam", two cut-offs (example numbers)

BOTH WORK THIS WAY

Jev, from TypeSafe
launched 15 September 2026

OpenAI's Decisions API
public beta, released 6 October 2026

`OpenAI says` about 10x faster than its Responses API

One call to Jev: a form, all questions at once

a chat model

A BLANK PAGE

> Well, it's hard to say. The customer seems a bit upset, but they were polite. I would say mostly yes, maybe...

ONE TOKEN AFTER ANOTHER



question 1, then 2, then 3: token by token

time →

YOU GET BACK **any text at all**

ANSWERS **can change the next one**

Jev

A PRINTED FORM

Is the customer angry? ☐ yes ☐ no

Which team? ☐ billing ☐ technical ☐ sales

EVERY QUESTION AT ONCE

Q1
Q2
Q3



in parallel, at the same time. A probability for every option you wrote.

YOU GET BACK **only your printed options**

ANSWERS **each one on its own**

One call, read left to right

an example: sorting a support ticket

your program sends
the state: a message about a double charge
the questions: which team? wants a refund?

→

Jev, by TypeSafe
fills in the form: typed answers, with probabilities

→

the answers come back

```
team: billing, with a confidence
wants a refund: a probability
```

your code decides: route to billing

Three kinds of question

TypeSafe calls them primitives.



Noul: yes or no

One number from 0 to 1, the chance of yes. 0.97 is almost surely yes. 0.5: yes and no equally likely.



Choice: pick one

One option from your list. You get the pick, a probability for every option, and one confidence.



Score: a level

Levels 0, 1, 2 that you describe. Torn between 1 and 2, half on each: Score = 1 × 0.5 + 2 × 0.5 = 1.5.

Pick the type whose answer your code can act on directly

Probability and confidence are different

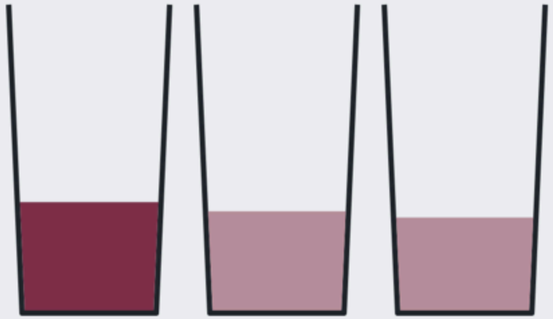
One answer, water shared over three glasses

Each glass is an option. Its water is that option's probability. All the water adds up to 1.



High confidence

All the water sits in one glass. One option clearly wins.

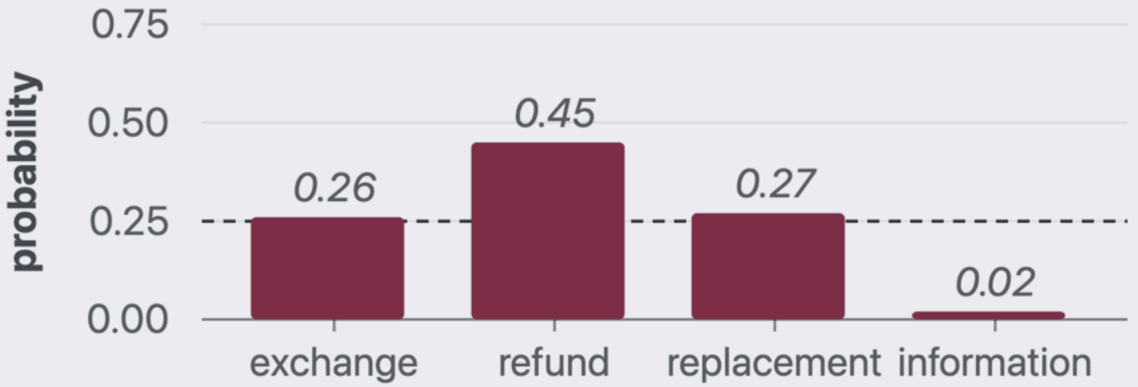


Low confidence

The water is shared out. The top glass still wins, but only just.

My run of script 06: "which resolution?"

```
python 06_probability_confidence.py
```



top probability (refund)
0.45

confidence from Jev
0.27

Dashed line: a pure guess, 1 in 4.
Confidence = (0.45 minus 0.25) ÷ 0.75 = **0.27**

Your numbers will move a little: read your own screen · confidence formula: TypeSafe docs

The same request, five times

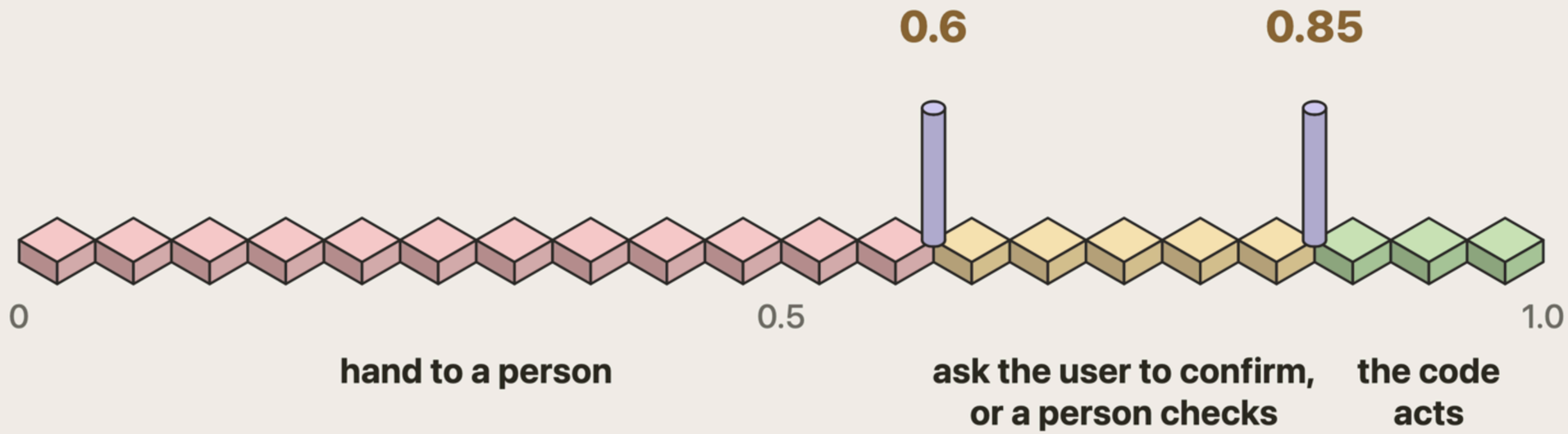
Refund was the top of four options every time. Its probability moved.



My run of 06_probability_confidence.py

Act, check, or hand to a person

TypeSafe's banking example: the two lines for moving money.



Jev's confidence in the answer, from 0 to 1

How to ask Jev well, and a hidden trick

`TypeSafe says` five tips from TypeSafe's docs

1	**Describe situations, not degrees**	“broken or degraded feature, but workaround exists”, not “moderately severe”
2	**Keep the options clearly different**	low confidence can mean two options overlap. Add “other” if needed.
3	**Put the facts in the material**	for a refund question, send the message and your refund policy
4	**Do arithmetic and dates in code**	“Jev is not a calculator.” Jev finds the date; your code compares
5	**Keep the material short**	“Include only the context relevant to the current questions.”

1 · literal: write the question you mean

2 · option order: keep options clearly different

3 · prompt injection: put the facts in the material

The hidden trick: a note inside the ticket `my run of 10_weak_spots.py`

clean ticket
`billing 1.00`
`confidence 1.00`

+ a note to the AI
`billing 0.99`
`confidence 0.98`

unclear ticket
`billing 0.71`
`confidence 0.56`

unclear + one calm sentence
`shipping 0.86`
`confidence 0.79`

Each card: the team Jev picked and that team's probability, then Jev's confidence, a different number. One calm sentence in an unclear ticket moved the answer to another team. Your numbers will move a little.

Where TypeSafe says Jev fits

Automation in software

Runs a million times in the background. Your code runs the steps; Jev judges.

we build: triage

Real-time decisions

About 150 ms, their docs say: fast enough for a screen or a game.

Very large datasets

Cheap enough to judge millions of records and find duplicates.

we build: duplicates

Checking other AIs

A cheap check for mistakes before an expensive answer is used.

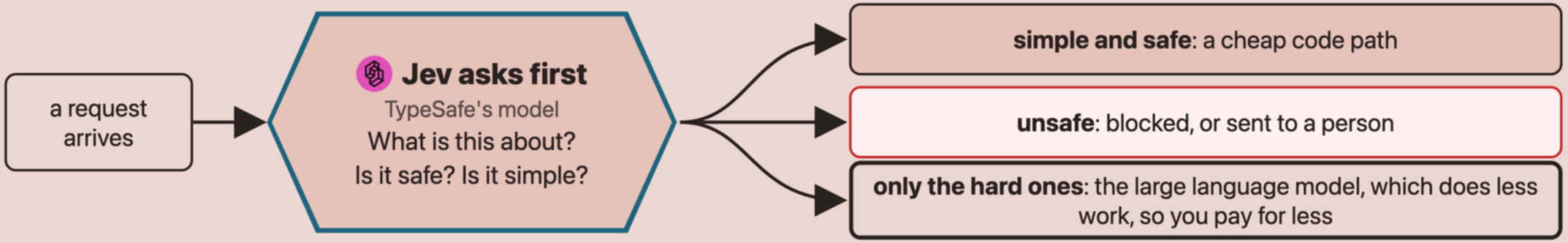
we build: a guardrail

Guiding an AI agent

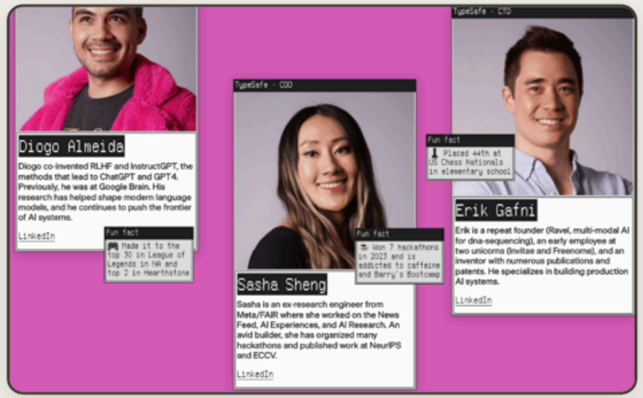
Choose the tool, or stop a bad step, before the big model runs.

we build: tool routing

Jev in front of the expensive model



Who built Jev, and why the name



typesafe.ai/team, three founders

DIOGO ALMEIDA, TYPESAFE'S CEO, LAUNCH POST, 15 SEP 2026

"Models have been superhuman at chat for years, so where is all the automation?"

Chat models were trained to chat. TypeSafe wants a model trained for fast, exact decisions in software.

> **"unstructured state in, typed probabilistic decisions out"** — Messy text goes in. Clean answers come out, each with a probability from 0 to 1.

A young company in San Francisco · typesafe.ai home and team pages, saved 4 Oct 2026

FROM TWO BLOG POSTS TO OPEN SIGN-UP IN SEVENTEEN DAYS

10 Sep blog post: The Bitterest Lesson → (1 day) **11 Sep** blog post: Lies, Damned Lies, And Benchmarks → (3 days) **14 Sep** first public Python library → (1 day) **15 Sep** launch, at first for only some people → (12 days) **27 Sep** waiting list ends: anyone can sign up

THE JEVONS PARADOX, 1865

> But the economy of coal in manufactures is a different matter.
> *It is wholly a confusion of ideas to suppose that the economical use of fuel is equivalent to a diminished consumption. The very contrary is the truth.*

econlib.org, The Coal Question

~~efficient engines, so less coal in total~~

"The very contrary is the truth." W. S. Jevons

history: coal, and it happens again each time engines improve

engines get more efficient → each job costs less coal → many more jobs worth doing: pumps, factories, trains → **total coal use goes up**

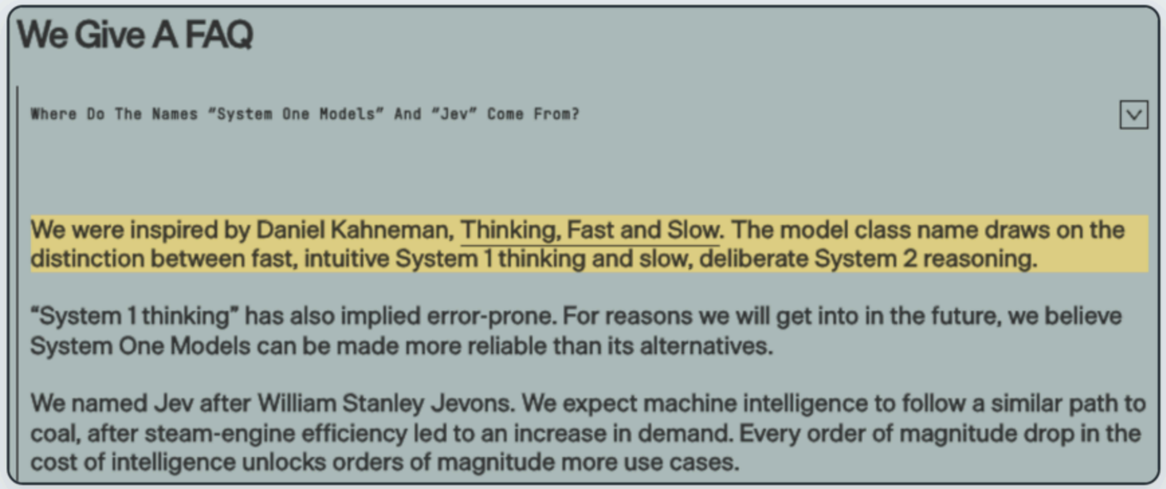
TypeSafe's expectation, not a fact, and why it named the model Jev

a decision gets about ten times cheaper → many more decisions worth automating → total use of AI decisions goes up

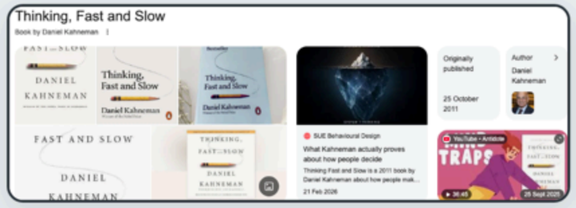
System One, from Kahneman's book

TypeSafe calls Jev a “System One model”.

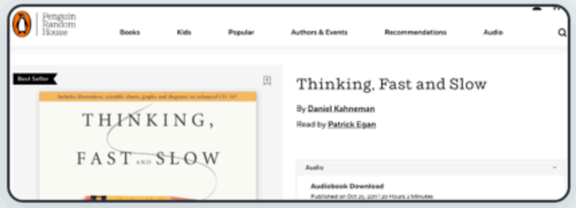
`TypeSafe says` The FAQ names the book, **Kahneman**, and the economist **Jevons**.



typesafe.ai/blog/introducing-system-one-models-and-jev



google.com, the book, 2011



penguinrandomhouse.com

Kahneman: System 1 is “fast, automatic, effortless, associative”.

SYSTEM 1	SYSTEM 2
fast and automatic	**slow, step by step**
four is simply there	you stop, work it out
2 + 2	17 × 24
in software	in software
Jev: one fixed question	**reasoning, token by token**

The bat and the ball: together they cost $1.10. The bat costs $1.00 more than the ball. How much is the ball?

fast answer
~~ball = 10 cents~~
bat $1.10, total $1.20, wrong

slow working
ball = 5 cents
bat $1.05, total $1.10, right

A puzzle by Shane Frederick. Kahneman's lecture says about half of the students asked at Princeton got it wrong.

Source: Kahneman, Nobel lecture, 8 Dec 2002 · labels: Stanovich and West, 2000

Two big numbers, then do the division yourself



typesafe.ai, saved 4 Oct 2026

`TypeSafe says` launch post: "on the higher end of real world gains"

TypeSafe AI	LLMs
Cost $0.000081	Cost $0.013880
Completed in 0.114s	Completed in 8.566s

the example under it, same page

`*Based On Workflows For System One Tasks (Proof)`

the star, enlarged

time, TypeSafe's example

8.566 s ÷ 0.114 s

= 75.1 about 75 times faster

the headline says 193.6x

cost, TypeSafe's example

$0.013880 ÷ $0.000081

= 171.4 about 171 times cheaper

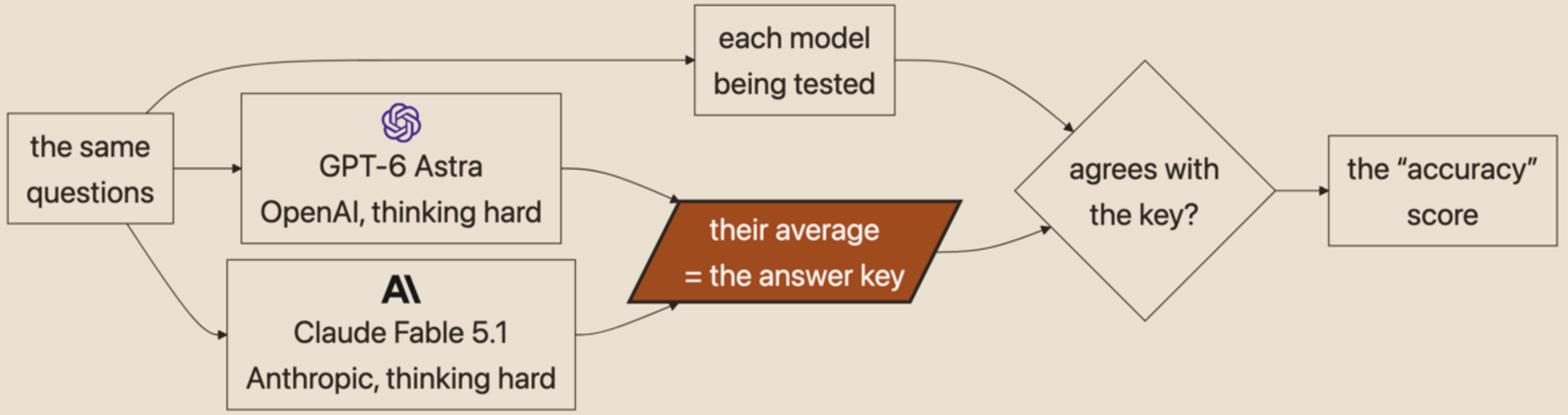
the headline says 444.6x

Not a mistake: the headline comes from a different comparison, their workflow tests.

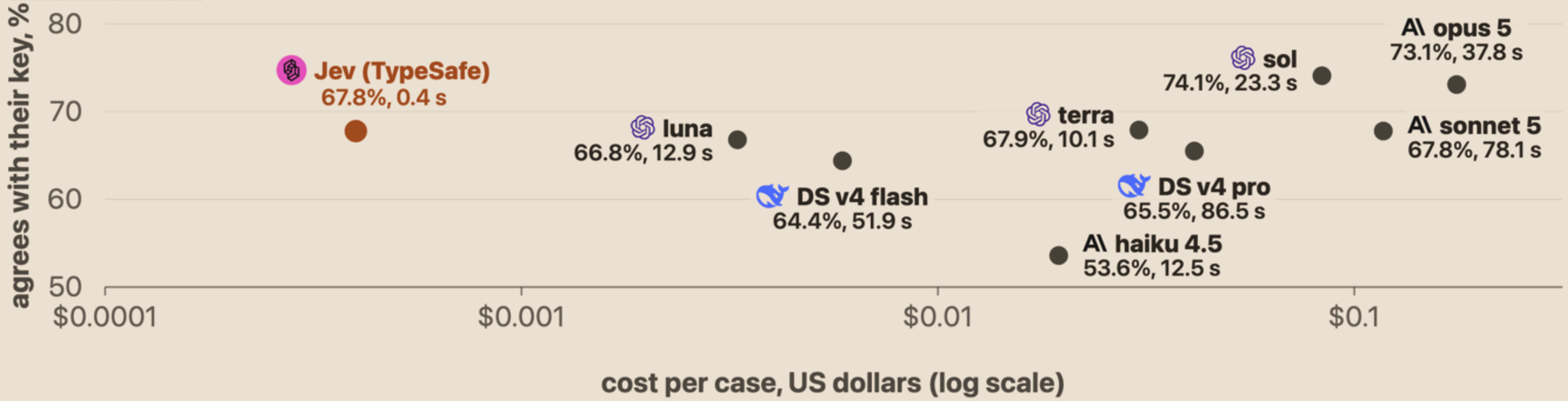
Source: typesafe.ai home page, saved 4 Oct 2026 · inputs: TypeSafe's own example · the division: mine

Who wrote the answer key, and the nine models

Four tasks. Here, accuracy means agreeing with two big models.



`TypeSafe says` **TypeSafe's own test with that key, their numbers. Higher means more agreement; further left means cheaper.**



Source: typesafe.ai/evals, saved 4 Oct 2026

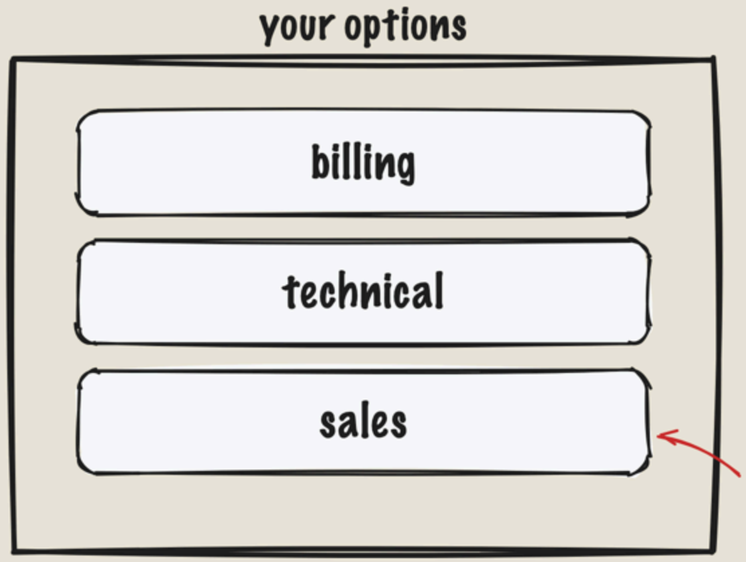
“Can't hallucinate” means the shape, not the truth

To hallucinate is to state something made up as if it were true.

home page: “Zero Hallucinations”
launch post: “can't hallucinate”

~~so Jev is always right?~~

No. It means the shape.



can it invent “partnerships”?
No: it is outside your list

can it pick “sales” when the answer is “billing”?
Yes: wrong, but in shape

TypeSafe FAQ: “Jev guarantees the shape of its answers, not that every decision is correct.”

Every claim, its test script, and a fair test

SCRIPT	WHAT IT COVERS	WHAT I FOUND, MY RUNS FROM KOLKATA
02 03 04	**three question types**	work as described; the Noul numbers were close to the docs
05	**many questions, one call**	far fewer tokens than one call per question
06	**confidence has a formula**	the Choice confidence matches the published formula
07	**stricter lines for risk**	a few lines of Python: act, ask, or a person
08	**model names and versions**	some model names from TypeSafe's own docs were rejected
09	**limits and errors**	the documented limits on Score levels and Choice options are real
10	**weak spots, tested**	literal words, option order, a trick inside the input
11	**a small support inbox**	my project: most teams right, unsure tickets to a person
12	**a small fair test, Jev against chat models**	30 hand-made tickets; needs an OpenAI key
13	**a short example to copy**	all three question types in one call

How the comparison was made fair same tickets, same questions, same machine, same time window

first, a warm-up: a few untimed calls to each model

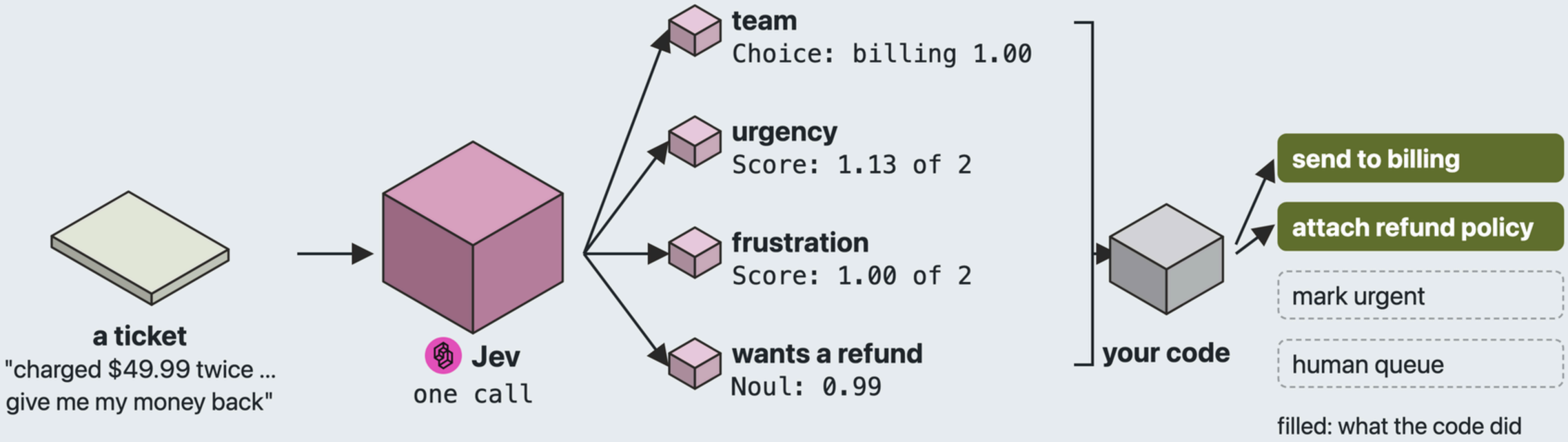
ticket 1 Jev → luna → sol

ticket 2 luna → sol → Jev

ticket 3 sol → Jev → luna

One call at a time. The first in line moves each ticket, so no model always gets the quiet network moment.

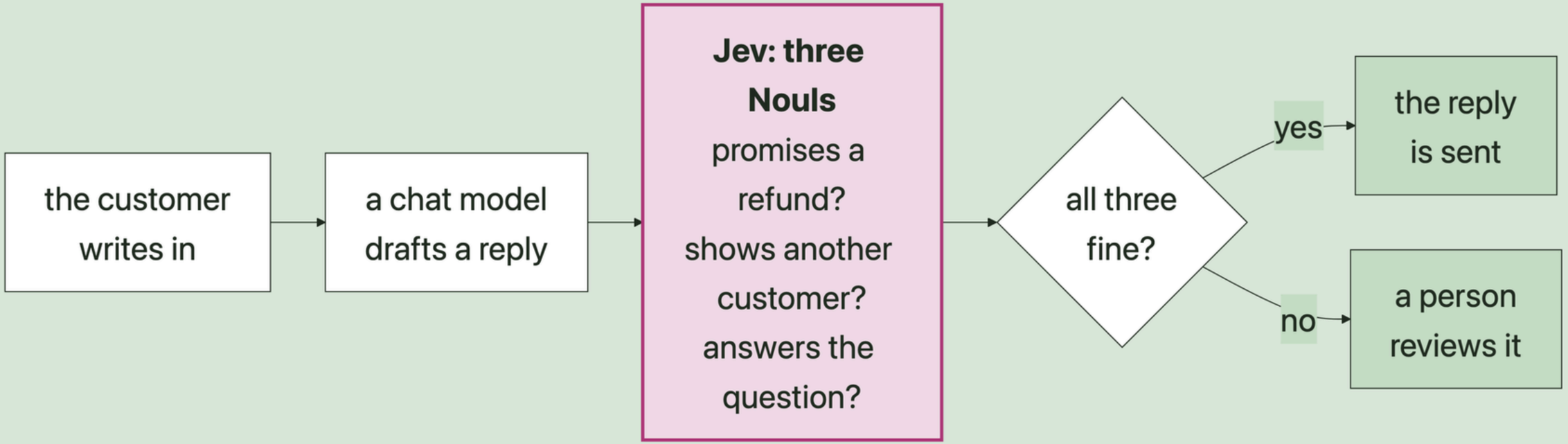
Ticket triage



Project 1, step by step `run from the repo root: python usecases/steps/<file>`

- **`project1_step1_four_questions.py`** — asks Jev four questions about one ticket in one call: team, urgency, frustration, refund
- **`project1_step2_route_tickets.py`** — turns the four answers into a route: team queue, urgent flag, refund policy

Checking a draft before it is sent

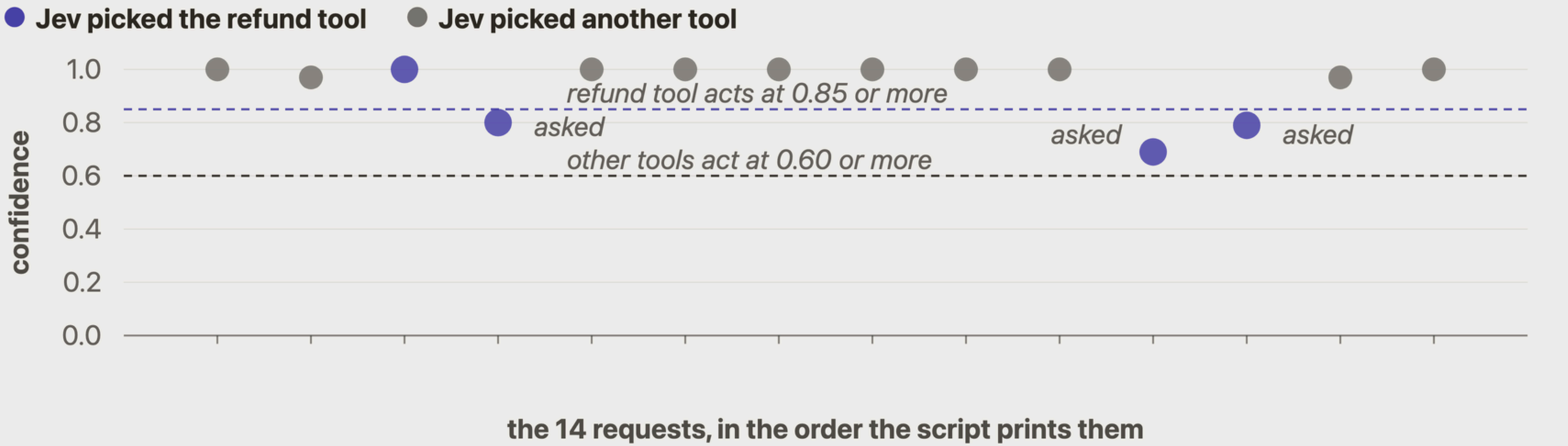


Project 2, step by step `run from the repo root: python usecases/steps/<file>`

📦 **`project2_step1_one_check.py`**	asks Jev one yes or no check about one draft reply
📦 **`project2_step2_three_checks.py`**	adds two more checks in the same call
📦 **`project2_step3_eight_drafts.py`**	runs the checks on eight drafts and blocks the bad ones
📦 **`project2_step4_live_draft.py`**	a chat model writes a live draft, and Jev checks it before it is sent

A stricter line for the tool that moves money

14 requests. Each dot is Jev's confidence in the tool it picked.



Project 3, step by step `run from the repo root: python usecases/steps/<file>`

`project3_step1_pick_a_tool.py`	asks Jev which tool a user request needs
`project3_step2_many_requests.py`	runs it on many requests
`project3_step3_money_rule.py`	adds a stricter line for tools that move money

The big project and its messy data

the Orchid records

ORCHID ENGG LTD
14 Market Lane, Leeds

Orchid Engineering Limited
14 Market Lane, Leeds

Orchid Engineering Ltd
14 Market Lane, Leeds

ORCHID CERAMICS PVT. LTD.
10 MG Ave, Pune

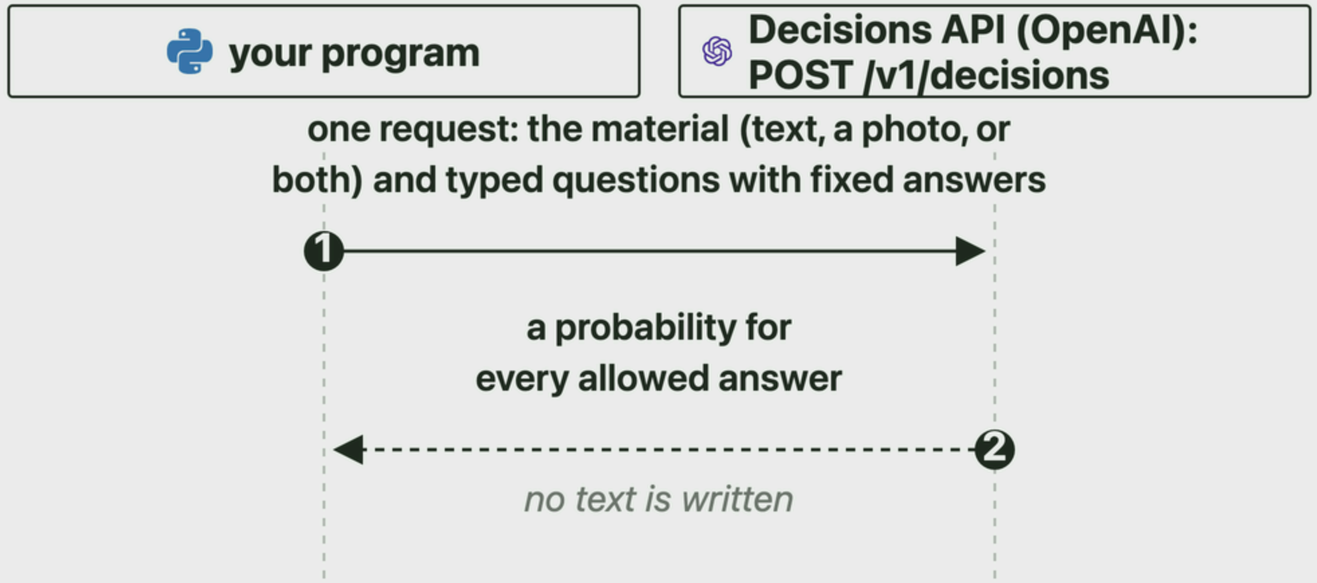
→ **every pair**
45 pairs

→ **Jev**
decides each pair:
same /
check /
different

→ **records joined into companies**

all in one file: `python usecases/big_project.py`

OpenAI's Decisions API, and when to use it



released: **public beta, 6 October 2026**

model: **one model, gpt-6-luna**

price: **$0.10 per million input tokens, output not billed**

`OpenAI's claim` OpenAI says: about 10x faster than its Responses API

Jev (TypeSafe) reads **text only**

the Decisions API (OpenAI) reads **text and photos**

USE THE DECISIONS API

when you need an answer from a fixed list

is this true? predicate: *is this photo damaged?*

which option? choice: *which team gets this ticket?*

how much, on ordered levels? score: *how bad is this bug?*

USE THE RESPONSES API

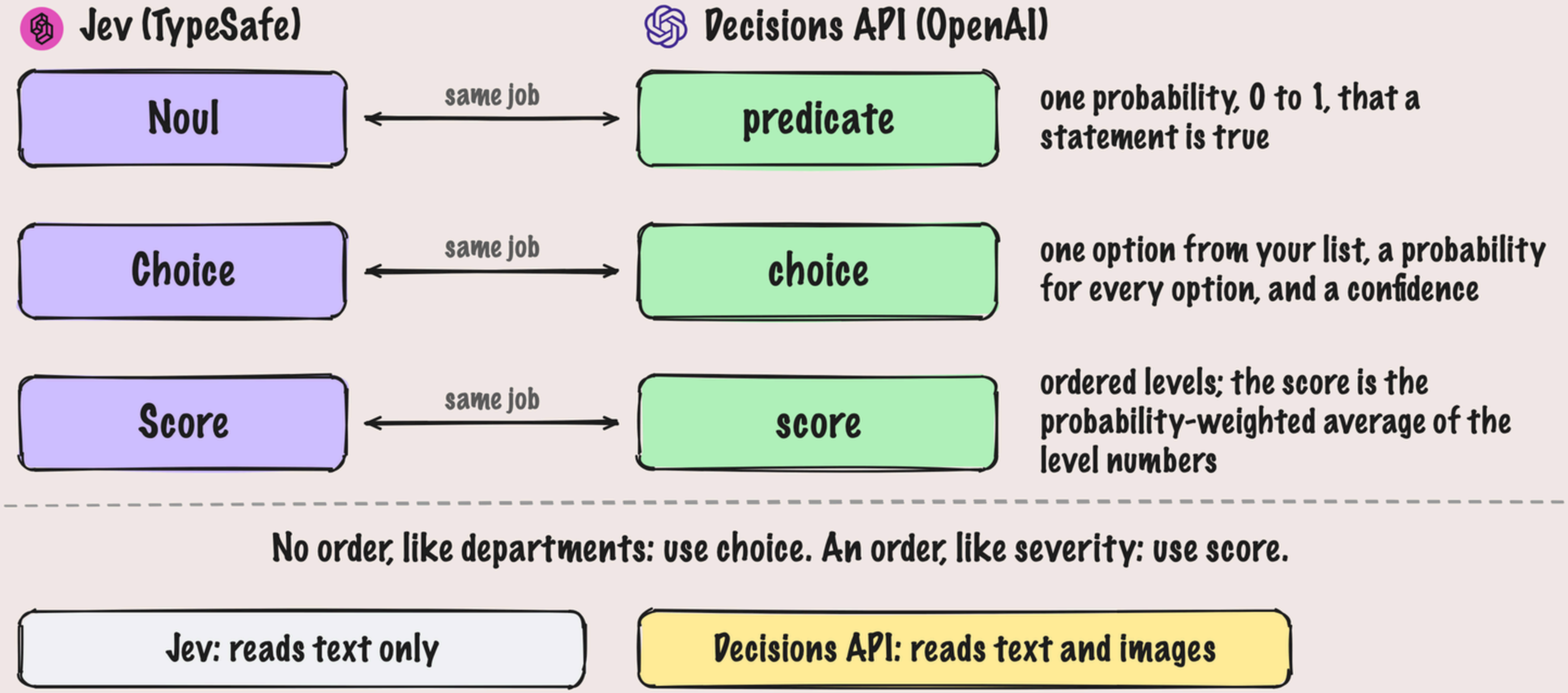
when the model must write

Structured Outputs: *JSON in your own layout, fields copied out of a document, or a written explanation*

function calling: *the model asks for one of your tools to be called*

Source: OpenAI's Decisions API guide, read 9 Oct 2026

The same three question types as Jev



Reading the raw answer, and what it cost

the raw JSON the Decisions API sends back, in three pieces

```
(.venv) decisions $ python 01_first_call_httpx.py
{
  "model": "gpt-6-luna",                    1
  "answers": [                              2
    {
      "type": "choice",
      "name": "department",                 3
      "choice": "billing",                  4
      "probabilities": [                    5
        {
          "value": "billing",
          "probability": 1.0
        },
        {
          "value": "technical",
          "probability": 0.0
        },
        {
          "value": "shipping",
          "probability": 0.0
        },
        {
          "value": "other",
          "probability": 0.0
        }
      ],
      "confidence": 1.0                     6
    }
  ],
  "usage": {
    "input_tokens": 145,                    7
    "input_tokens_details": {
      "cached_tokens": 0,
      "cache_write_tokens": 0
    },
    "output_tokens": 0,                     8
    "output_tokens_details": {
      "reasoning_tokens": 0
    },
    "total_tokens": 145
```

1. **model**: gpt-6-luna answered
2. **answers**: a list, one per question
3. **name**: department, the question's name
4. **choice**: the winner, billing
5. **probabilities**: billing 1.0, technical 0.0, shipping 0.0, other 0.0
6. **confidence**: 1.0
7. **input_tokens**: 145, what you pay for
8. **output_tokens**: 0, nothing written

> Jev keys its answers by name (`"answers": {"urgency": ...}`). The Decisions API sends a list, in the order you asked.

What this one call cost

7 `145 × $0.10 ÷ 1,000,000 = $0.0000145`

the script prints: `cost (US$) 0.00001450`

8 output tokens: 0, so nothing to add

In plain words: about one and a half thousandths of one US cent.

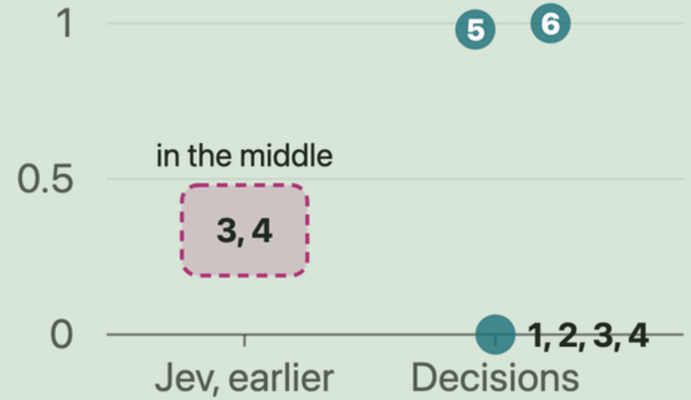
OpenAI's guide: some requests cost more, such as very long inputs. Ours were short. The scripts in this course use the plain price.

Yes or no, and a score between two levels

Yes or no The probability that the customer is asking for a human. Jev calls it a Noul; the Decisions API calls it a predicate.

	MESSAGE	Decisions
1	Thanks, that fixed it!	0.00
2	How do I reset my password?	0.00
3	I need this sorted today, whatever it takes.	0.00
4	Are you a bot?	0.00
5	Is there any way to speak to someone about my invoice?	0.98
6	I have asked three times now. Can I please just talk to a real person?	1.00

The Decisions API's numbers sit at the two ends. Set the cut-offs for each one from labelled examples from your own application (OpenAI's guide).



A score "The export button crashes the settings page in Safari." Level numbers start at 0.

Decisions API: the probability of each level

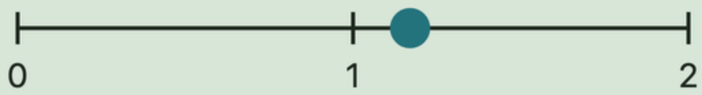


the score, worked out

```
0.04 × 0
+ 0.75 × 1
+ 0.21 × 2
= 1.17
```

confidence 0.63

the score on the 0 to 2 line

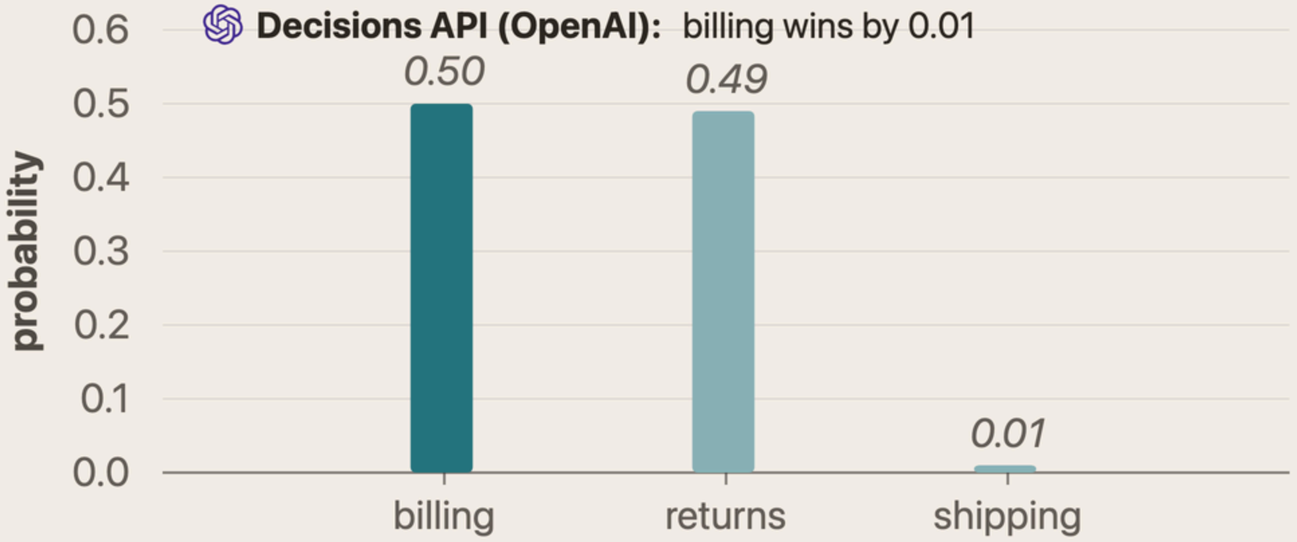


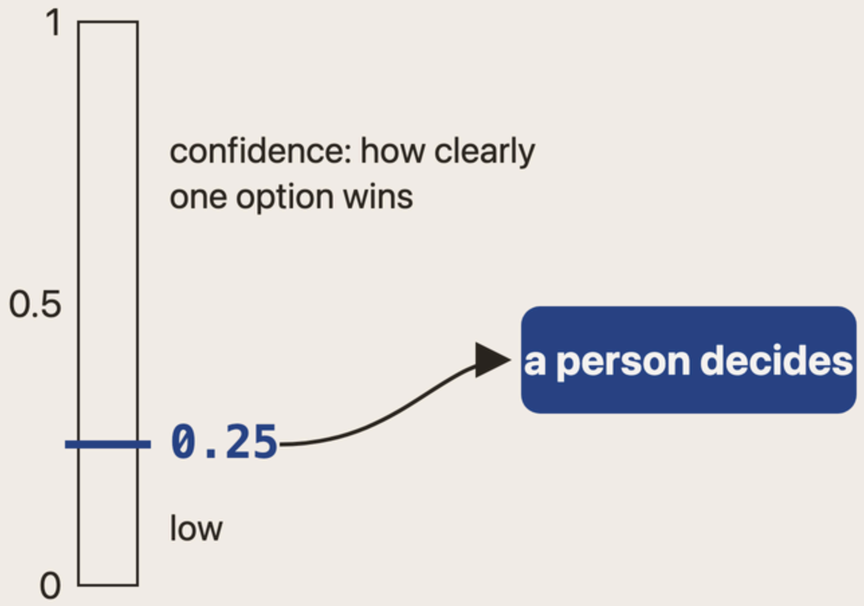
Decisions API (OpenAI): **1.17**

Jev (TypeSafe), earlier: this report also landed between the two levels, nearer the middle

A ticket that belongs to two teams

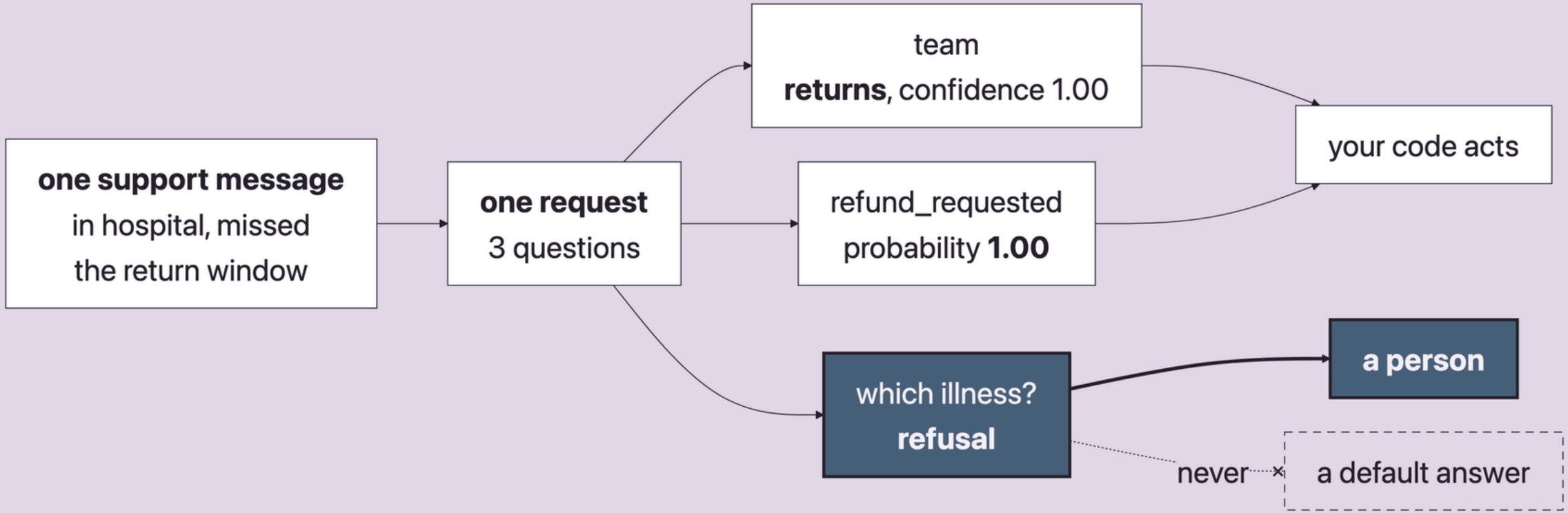
"Shoes arrived two weeks late and in the wrong size. Also I see two charges of $120 on my card."





Jev (TypeSafe) gave this same ticket a low confidence too, earlier in the course.

A refusal goes to a person



API reference: "The model declined to answer this question. Other questions in the same request can still receive answers."

My test: the same illness question with an "other" option was not refused. It answered "other", confidence 0.99. Send that to a person too.

Not the same as an error. An HTTP 400 fails the whole request, with no answers at all.

33 egg photos, and how each one is sent



Frame colour = my label: clean 18 dirty 12 cracked 3
`eggs/labels.csv`

Cut from 8 Wikimedia Commons photos by Burnerburnerburner8, safaritravelplus, Achim, Sanqui, Aline Ponce, 01x07x2022000, Garitzko and Paolo Neo. 6 CC0 and 2 public domain. Licences checked on 9 October 2026.

Labels: the course author's draft labels, made by eye. Not an expert's grades.

your code to Decisions API

1. read the bytes of the photo file
2. base64: bytes become text
3. prefix ``
4. one request: input_text, input_image, 3 questions
5. 3 answers come back

input_text: "One egg from a grading line." **questions:** choice `condition`, predicate `visible_crack`, score `dirt_level`



`box10-03` · clean 0.75
data URL 9,107 characters, 494 input tokens



`duck-04` · dirty 1.00
data URL 30,255 characters, 523 input tokens



`broken-02` · cracked 1.00
data URL 57,775 characters, 650 input tokens

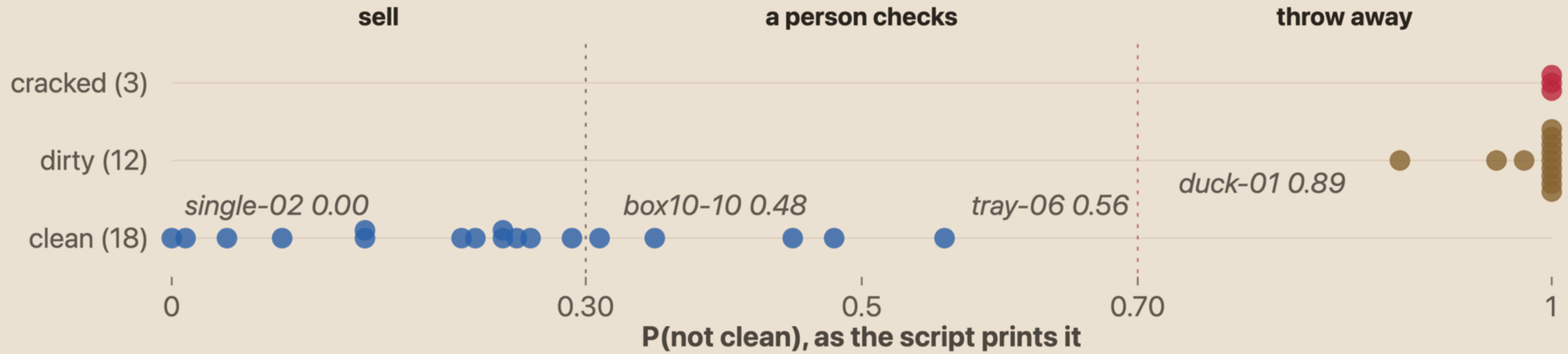
OpenAI's guide: base64 data URLs only. The API reference also allows public HTTP(S) URLs. This course always sends base64. Photos: Wikimedia Commons, CC0 and public domain.

How sure, egg by egg, and one rule

Total Technology Zonne

Decision AI Models
Jev and the OpenAI Decisions API

clean **dirty** **cracked** my labels · 33 calls, 8120 input tokens, cost $0.000812



13
sell
P(not clean) below 0.30

5
a person checks
between 0.30 and 0.70

15
throw away
P(not clean) above 0.70

0
mistakes
bad egg sold, clean egg thrown

A person checks:
`tray-06` P(not clean) 0.56 (my label: clean)
`box10-01` P(not clean) 0.35 (my label: clean)
`box10-08` P(not clean) 0.31 (my label: clean)
`box10-09` P(not clean) 0.45 (my label: clean)
`box10-10` P(not clean) 0.48 (my label: clean)

My own labels, made by eye: a demo, not a proof.

The refund desk: four steps and one rule

The four step files

`from decisions/: python steps/<file>`

- **`desk_step1_one_photo.py`**
  sends one photo and one question
- **`desk_step2_add_message.py`**
  adds the customer's message
- **`desk_step3_policy.py`**
  adds the refund policy and the send-to-a-person path
- **`desk_step4_all_claims.py`**
  runs the desk over all the claims

The shop's rule, in order

the message is not about damage	**a person checks**
the message says broken or dirty, and the photo agrees at 0.90 or more	**refund now**
anything else	**a person checks**

The shop never says no on its own. Every no is a person.
A real shop would also cap each refund and check for reused photos.

`DESK_STEP4_ALL_CLAIMS.PY, TWO CLAIMS TO LOOK AT`



C03 · box10-03 · the clean egg from the start; the message says cracked
message says **broken**, photo shows **broken**, photo agrees **`0.66`** · **a person checks**



C04 · duck-01 · borderline dirty, right next to the line
message says **dirty**, photo shows **dirty**, photo agrees **`0.92`** · **refund now**

All 8 claims: 5 refunded now, 3 for a person to check · $0.000346

Why C03 matters

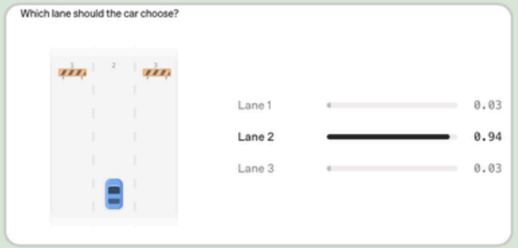
The egg in the photo is clean, but the customer's words pushed the photo answer toward broken. Asking about the photo in its own call stops that.

That version: `steps/extra/desk_step5_photo_alone.py`

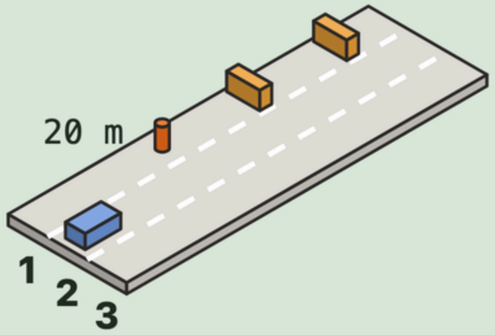
When money depends on the photo, ask about the photo on its own.

Photos: Wikimedia Commons, CC0 and public domain.

From OpenAI's lane example to our lane race



OpenAI's lane example is an illustration on its page, not a live run: nothing is sent to the API. Next, a real race you can run.



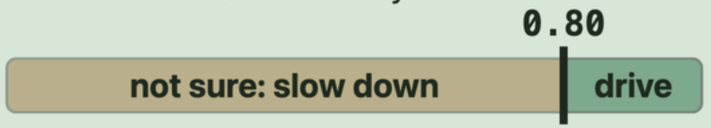
Seed 7, step 0 (`road.json`). Our car (blue): lane 2.

Jev or the API answers one choice

lane 1, 2 or 3? Three probabilities come back.

Our code decides

The most likely lane at 0.80 or more: drive into it. Below that line, or no answer: slow down and stay.



Something in the lane at 20 m is a crash. Slowing down never crashes here; it only costs time.

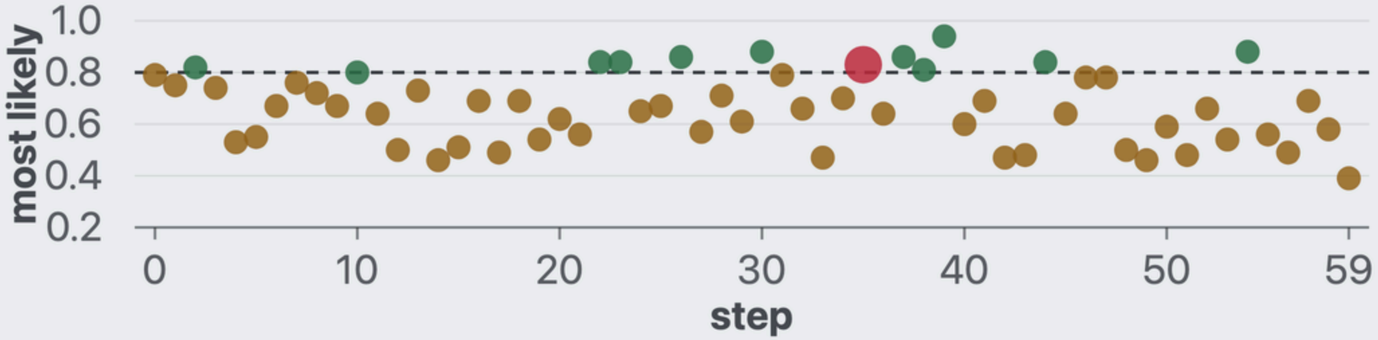
The files you run, in order, from `decisions/`

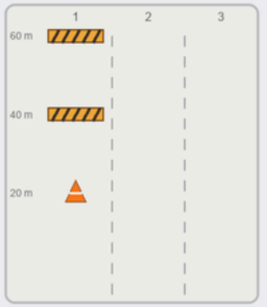
`steps/race_step1_one_step.py`	one saved picture of the road, one lane choice
`steps/race_step2_full_race.py`	all 60 steps; drive on only at 0.80 or more, else slow down
`steps/race_step3_video.py`	the same race from video frames; add 3 for three frames

From a picture or video frames, rarely sure

● **drove on: 11** ● **slowed down: 48** ● **crashed: 1**

One saved picture per step (`steps/race_step2_full_race.py`), 60 steps. Each dot: the top lane's probability; dashed: the 0.80 line.





`race_images/step_00.png`
Blocked at 20 m: lane 1 (a cone). Lane 2 was clear and got 0.79, just under 0.80, so the car slowed down. Not sure is not wrong, but it costs time.

a video file
`race_images/clip.mp4`
244 frames

→

cut into frames once
with ffmpeg, saved in
`race_images/frames`

→

step 10: frame 40, or 3



38, 39, 40, oldest first

→

each frame is sent as one image
all in one request

→

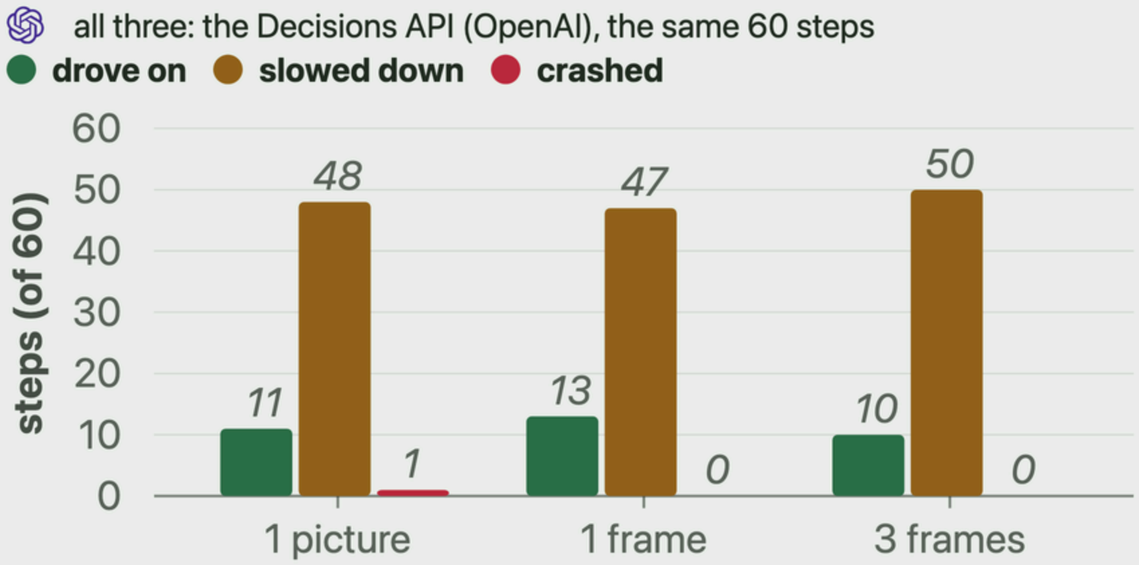
Decisions API
answers one choice question

60 steps, the 0.80 line	1 picture	1 frame	3 frames
drove on	11	13	10
slowed down	48	47	50
crashed	1	0	0
cost, 60 steps	$0.0022	$0.0023	$0.0040

$0.0040 against $0.0023: 1.7 times the cost, and no help. A cone stands still, so only where it is right now matters. Send more frames only when the answer depends on movement (not tested here).

No video type. The API reference: "up to 128 images per request". "Other message roles, function calls, files, audio, and item references aren't supported."

Three runs, and what the 0.80 line cannot do

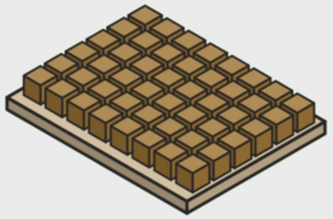


60 steps each, the 0.80 line.

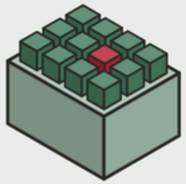
RUN	SECONDS	COST
one picture per step	**20**	**$0.0022**
one video frame per step	20	$0.0023
three video frames per step	23	$0.0040

In this game, slowing down never crashes and the game waits for every move. A real car does not wait.

The picture run's 60 steps, one block each



below 0.80: 48 steps, and the car slowed down on every one. Not sure costs time, not a crash.



0.80 or more: 12 steps, 11 drove on, **1 crash** in this run.

TRUE FOR THIS ONE ROAD

1. From pictures the model was rarely sure: the car drove on only 11 times out of 60.
2. A high number can still be wrong: the crash came from above the line.
3. More frames cost more and did not help here.
4. Keep a safe action for the not-sure band, and plain checks around the model. Check that the safe action really is safe.

A fair test: 30 tickets, the same questions

`compare_decisions/side_by_side.py` ticket 1 → Jev → Decisions API; ticket 2 → Jev → Decisions API; to ticket 30 **one call at a time**

The same 30 support tickets (every tenth of `data/tickets.jsonl`) and right answers, **the same 3 questions** word for word in one call: team, urgency, frustration. **One call at a time:** Jev, then the Decisions API, ticket by ticket. **No retries:** a slow call keeps its time.

How to read the results · read every number from your own terminal

COLUMN	WHAT IT MEANS
team · urgency · frustration	right answers out of 30: answers that match the labels; urgency and frustration come back as scores, rounded to the nearest level
seconds	the average time of one call, one at a time, with no crowd
cost	all 30 calls: the input tokens in each answer, times the price per token
tokens per ticket	how much each API counted for one ticket and the questions; each counts in its own way

> Two rows: **Jev** (jev-1.13.0) and **Decisions API** (gpt-6-luna). These are made-up tickets, where one program wrote both the tickets and their labels: a result, not a promise.

Which one for which job, and both together

only the Decisions API (OpenAI)
- reads photos: it takes text, images, or both
- can refuse one question; the others in the same request are still answered

both
- many independent questions in one call
- a probability for every option
- a confidence number for a choice or a score

only Jev (TypeSafe)
- the state can be a JSON object with named fields, like a form: “String, JSON object, or array of text values”
- text only: “No image, audio, or video input”

Sources: docs.typesafe.ai/models · OpenAI's Decisions docs · 9 Oct 2026



Decisions API (OpenAI)
decisions about photos, or text with photos

of these two, the only one that reads images



test Jev and Decisions
decisions about text only

on our tickets Jev was the cheaper one; the Decisions API got more urgency and frustration answers right



chat model, structured output
written text for a person, or an answer in your own JSON shape

neither Jev nor the Decisions API writes text; OpenAI's Decisions guide says the same

Both together: a fast model or API with fixed options in front. The costly step only for the unclear ones.

a request (a ticket, a message) → **in front:** Jev (TypeSafe) or Decisions API (OpenAI) → how confident is the answer?

high → your code acts

low → moves on: a larger model, or a person

Text: decide by what a wrong answer costs you, then test on your own tickets. Whatever you pick: two confidence lines, the unsure middle to a person.

Before a big job, check your rate limits: OpenAI's Decisions guide does not state one for this endpoint; TypeSafe publishes Jev's on its models page. The Decisions API is a beta, so plan for when it is down: queue the work for later, or send it to a person.

Five things to remember

1 **Both answer typed questions with probabilities.**
A yes or no, one option from a list, or a level on a scale. Ask all your questions about the same material in one call: every call repeats some text, and in one call you pay for it once. One exception: when money depends on a photo, ask about that photo on its own.

2 **Wording matters on both.**
Write the question you mean. Write each option as you would explain it to a new colleague.

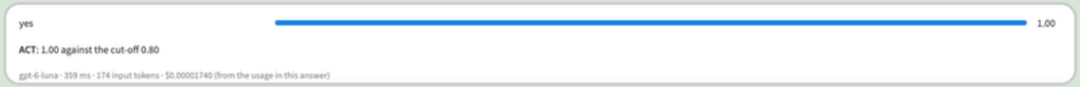
3 **Use the confidence.**
Two confidence lines, the unsure middle to a person, a stricter line for anything that moves money.

4 **Log which model answered every call.**
On Jev, pin jev-1.13.0. Luna has no dated version to pin, so re-check your confidence lines when OpenAI updates it.

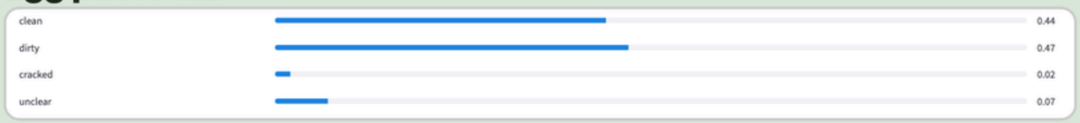
5 **Test on your own labelled data.**
Tune your questions on one set, then check them on a set you never used for tuning.

The app: three tabs, and when it makes a call

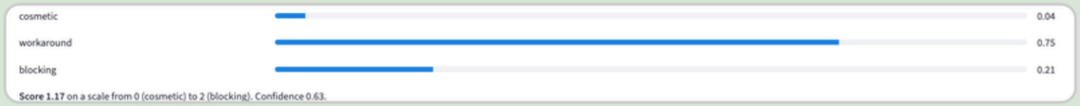
Yes or no tab



Egg photo tab



Score tab



a click on Ask

Streamlit runs the whole script again

↓

st.button("Ask") is true: **call the API**

↓

save the answer in st.session_state

↓

compare with the slider's line, draw the page

a slider move

Streamlit runs the whole script again

↓

st.button is false: **no call, no cost**

↓

read the saved answer from st.session_state

↓

compare with the new line

st.session_state keeps the saved answer: a rerun does not wipe it.

The files you run, in order, from the repo root, each with `streamlit run`

`app/steps/app_step1_page.py`	a page with one text box, no API call
`app/steps/app_step2_yes_no_tab.py`	the yes or no tab, with a slider for the line
`app/steps/app_step3_egg_photo_tab.py`	the egg photo tab, with two confidence lines
`app/steps/app_step4_score_tab.py`	the score tab

Thank you

All the code

every script, every project in steps, and the web app

Data and results

the programs that make the data, and every raw result

Thank you

to freeCodeCamp, for publishing this course free for everyone

Roni Das

systemdesign.academy and YouTube: Total Technology Zonne

github.com/ronidas39/jev-system-one-course